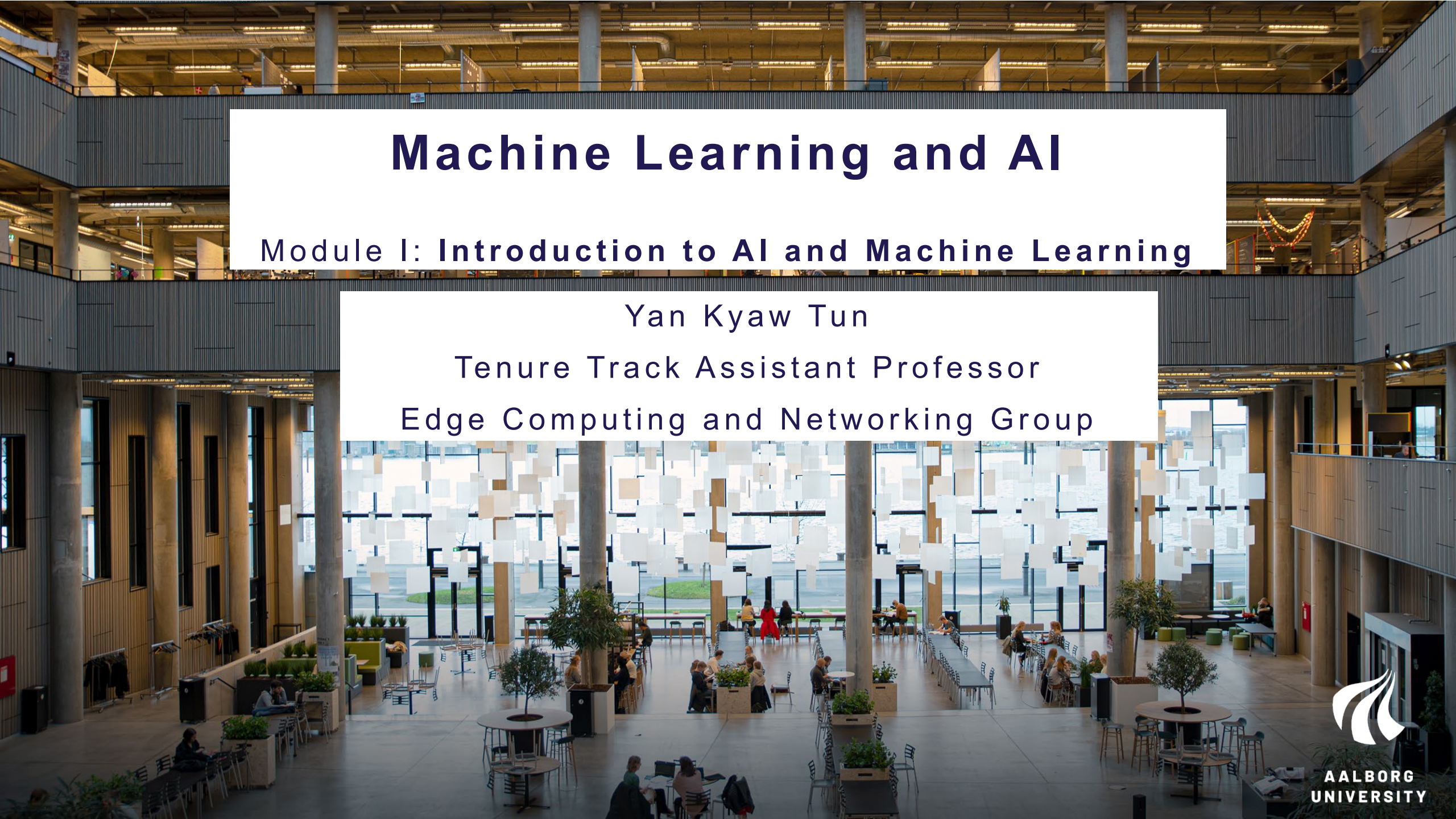


Machine Learning and AI

Module I: **Introduction to AI and Machine Learning**

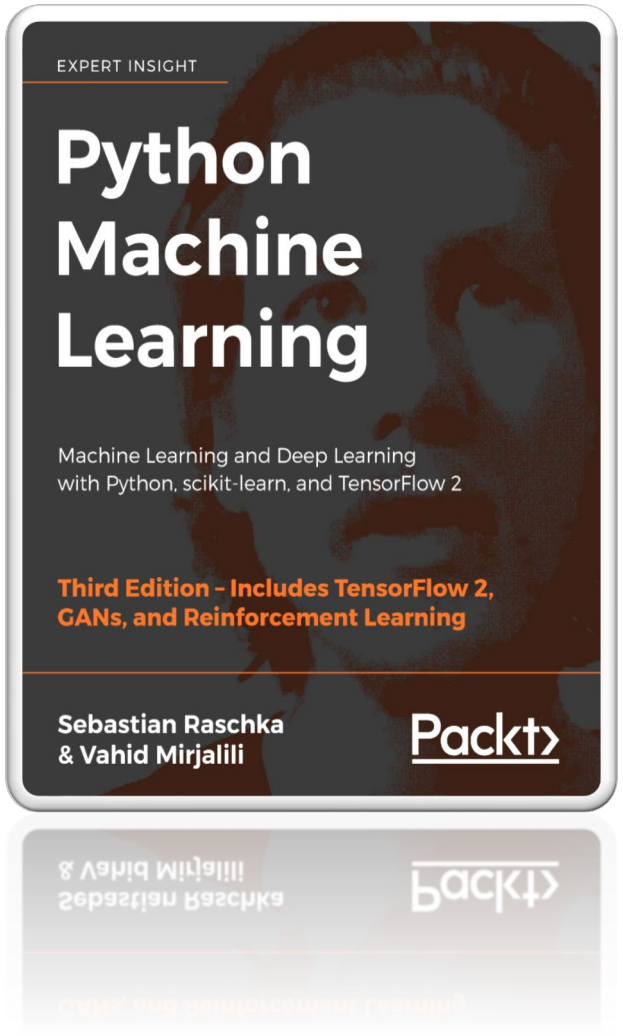
Yan Kyaw Tun

Tenure Track Assistant Professor

Edge Computing and Networking Group

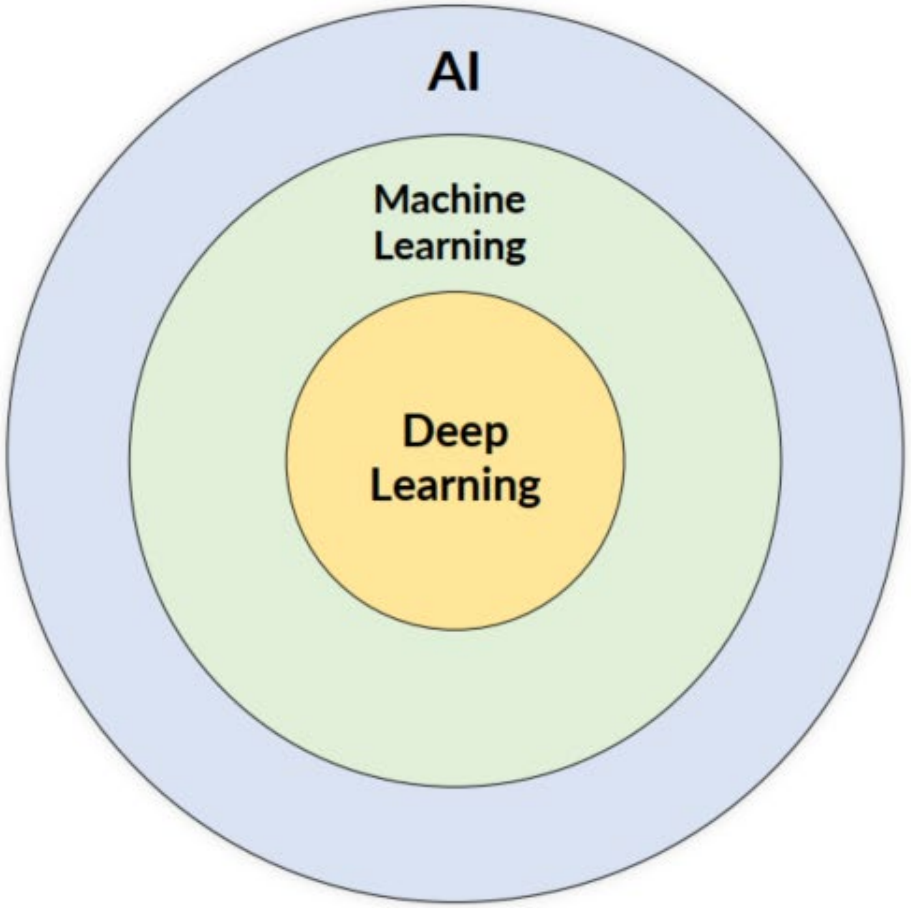
AALBORG UNIVERSITY

Text Book


EXPERT INSIGHT
Python Machine Learning
Machine Learning and Deep Learning with Python, scikit-learn, and TensorFlow 2
Third Edition – Includes TensorFlow 2, GANs, and Reinforcement Learning
Sebastian Raschka & Vahid Mirjalili
Packt

Artificial Intelligence (AI)

- Artificial Intelligence (AI) is the science and engineering of making machines as intelligent as humans



AI – Any technique which enables computers to mimic human behavior.

ML – Subset of AI techniques which use statistical methods to enable machines to improve with experiences.

DL – Subset of ML which makes the computation of multi-layer neural networks feasible.

http://www.deeplearningbook.org/contents/intro.html

3 Stages of AI

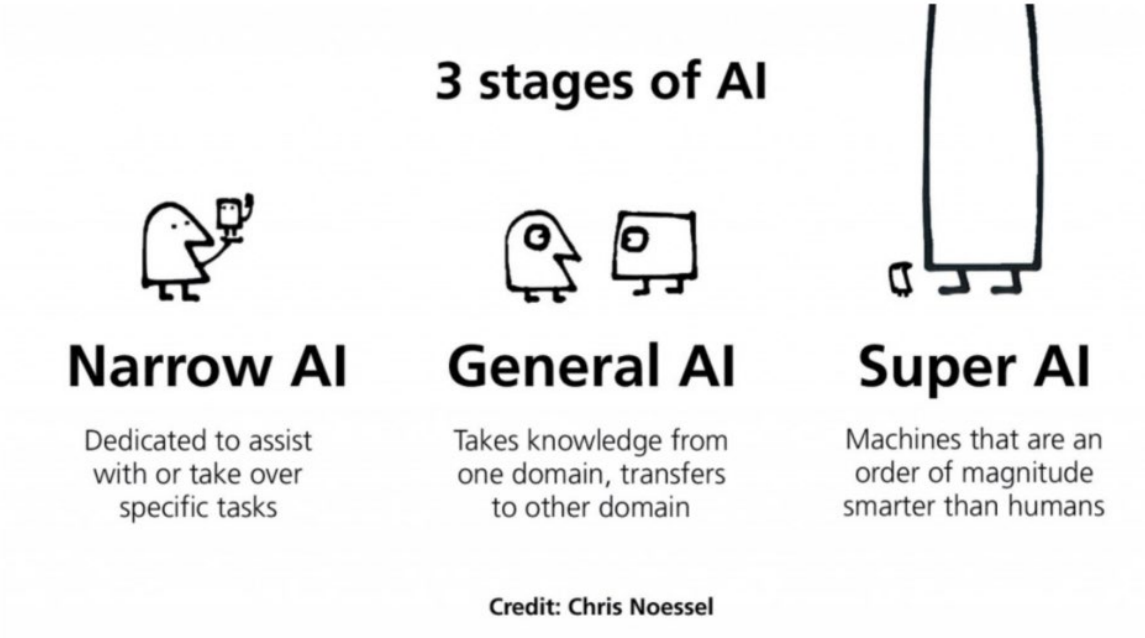


Image Source: https://www.datakeen.co/wp-content/uploads/2020/01/3_stages_of_ai.png

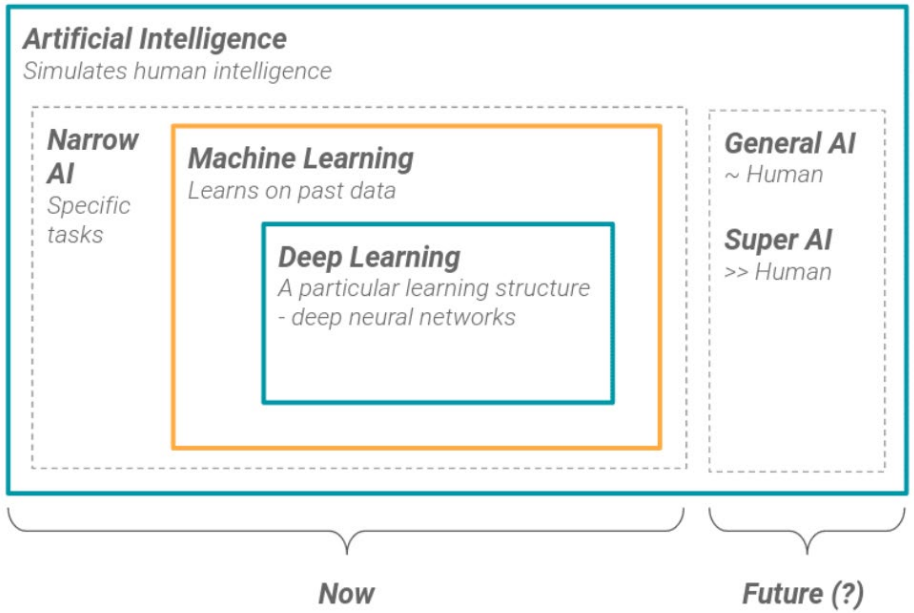
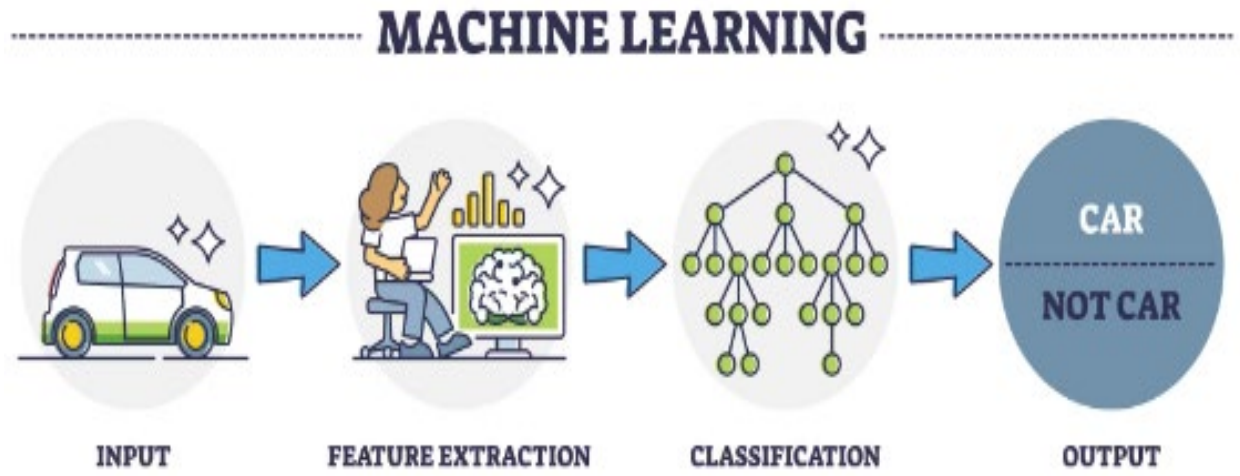


Image Source: https://www.datakeen.co/wp-content/uploads/2020/01/what_is_ai_summary_chart.png

Machine Learning

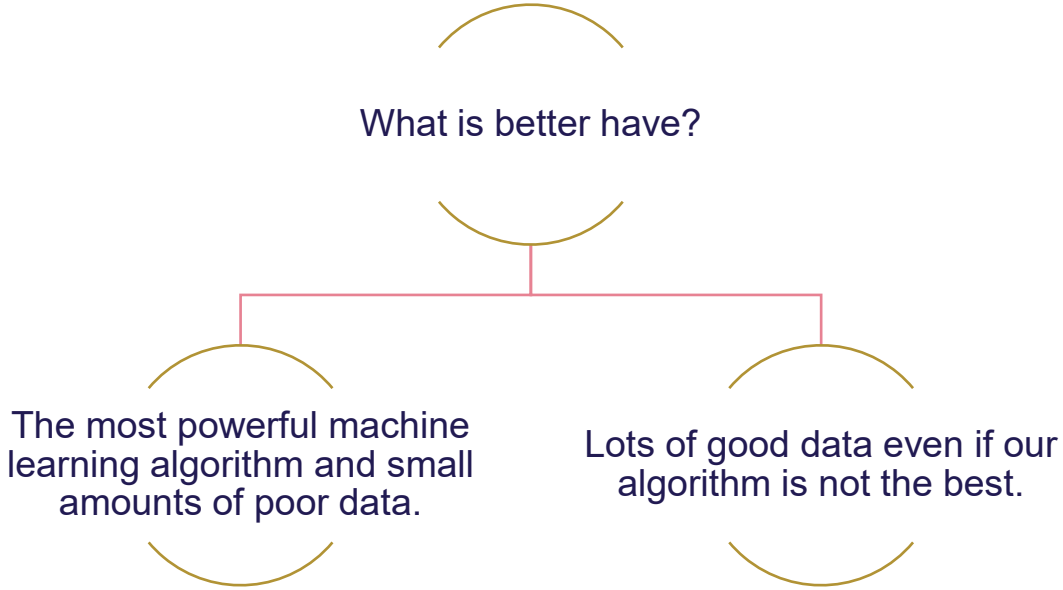
- Machine learning is a branch of artificial intelligence that enables computers to learn and make predictions or decisions without being explicitly programmed.

- It involves developing algorithms that learn patterns and relationships from data and use them to make informed decisions or predictions.



Data

- The algorithms are data hungry and data-driven. Thus, without data, we can't do anything

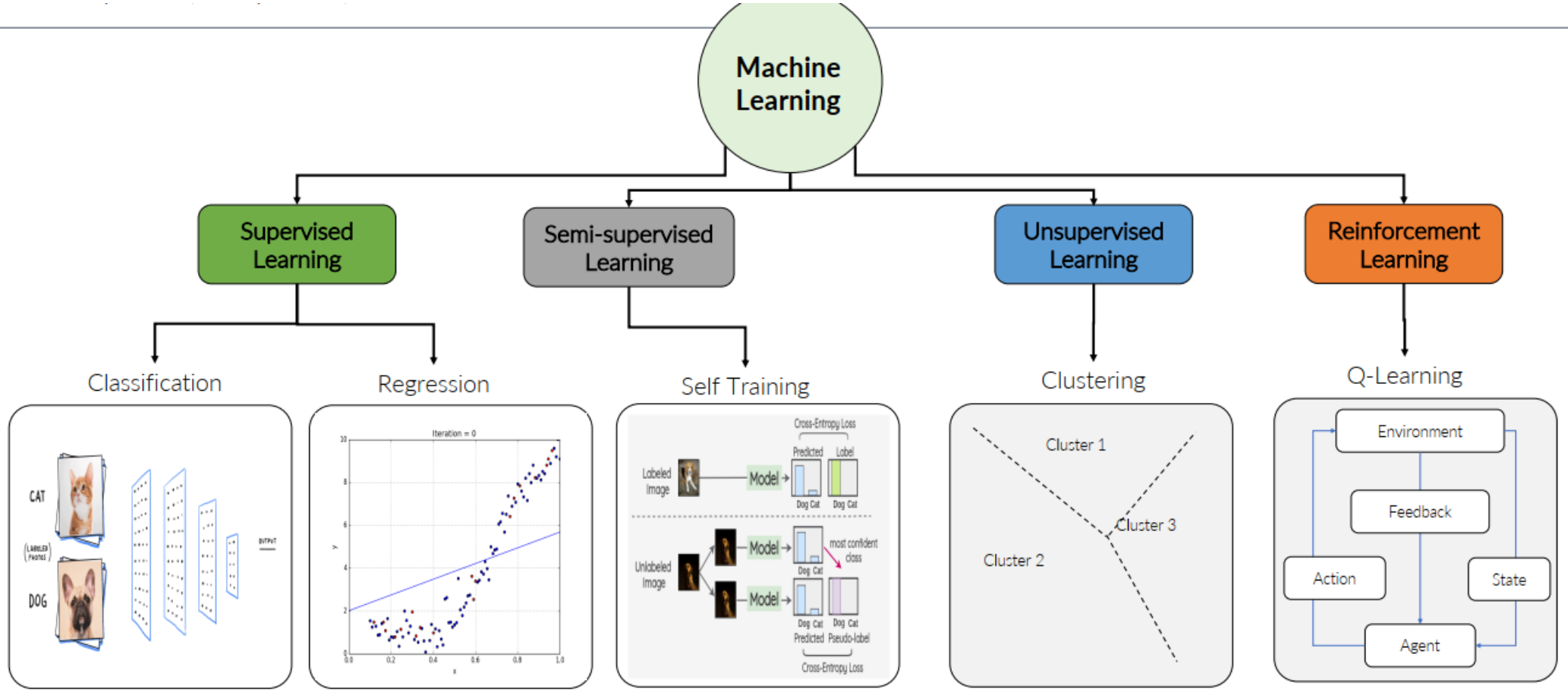


Why have companies like Facebook, google, etc. do so well?

Ans:

Because all of you that have social media feed them new data all the time and you even annotate it for them by labeling it with likes, etc.

Type of Machine Learning

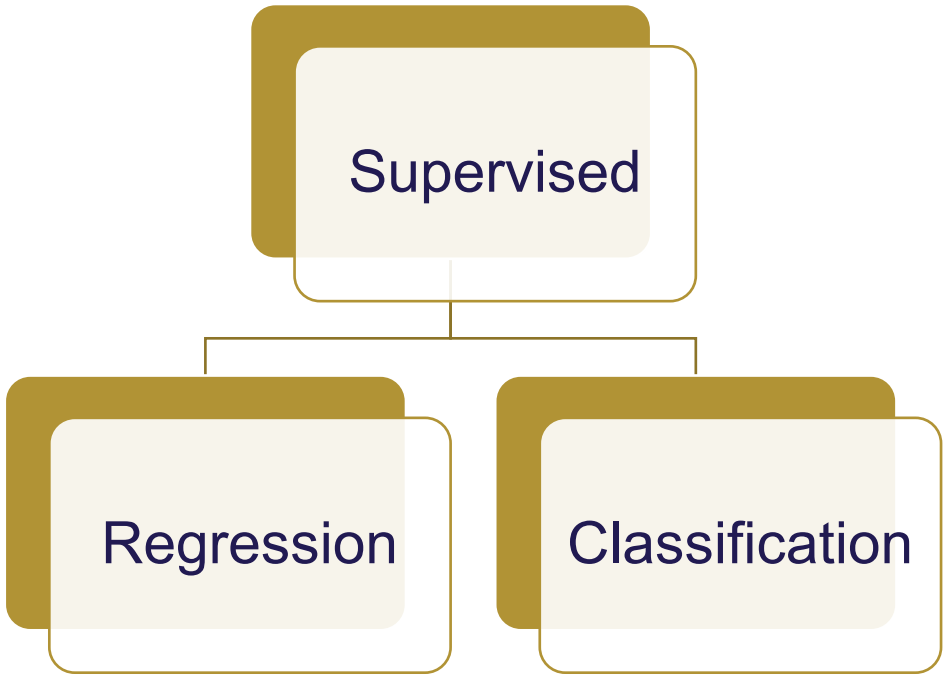


- Image Sources: https://github.com/ReiCHU31/Cat-Dog-Classification-Flask-App
- https://medium.com/analytics-vidhya/simple-linear-regression-cost-function-gradient-descent-50c5ed085770
- https://amitness.com/2020/07/semi-supervised-learning/

Supervised Learning

- The term supervised refers to a set of samples where the desired output signals (labels) are already known

- The main goal in supervised learning is to learn a model from labelled training data that allows us to make predictions about unseen or future data



Supervised Learning

Duck

Duck

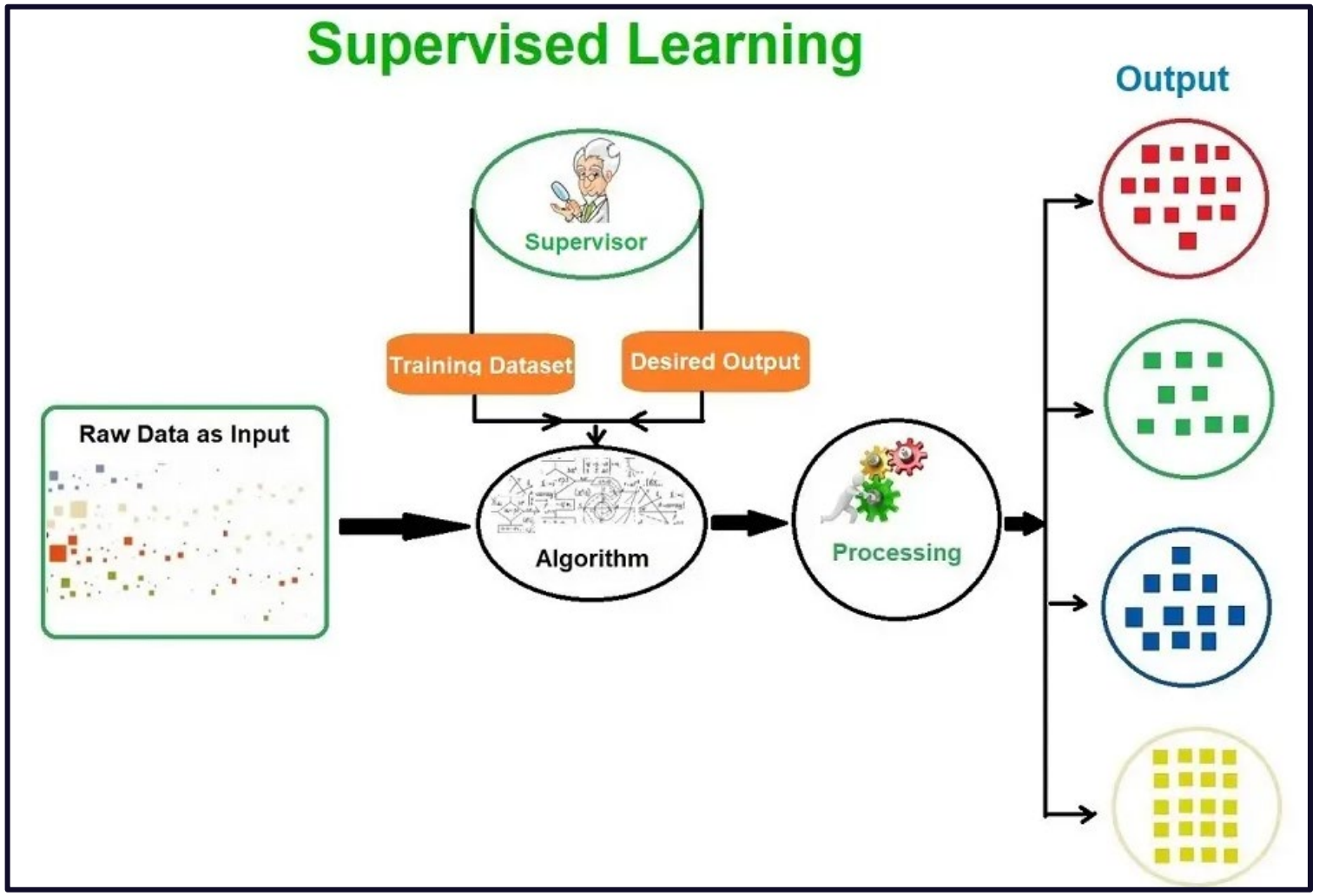
Predictive Model

Not a Duck

Predictive Model

Not a Duck

Supervised Learning



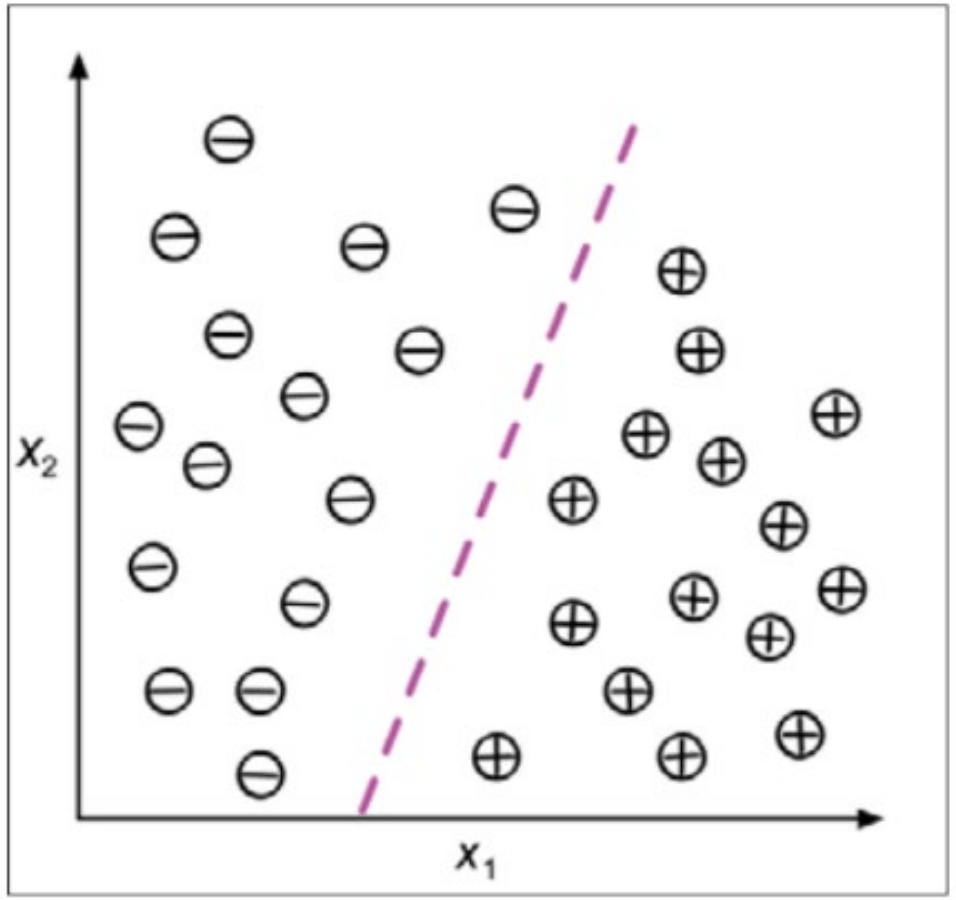
Source: https://www.ejable.com/tech-corner/ai-machine-learning-and-deep-learning/types-of-machine-learning/

Supervised Learning

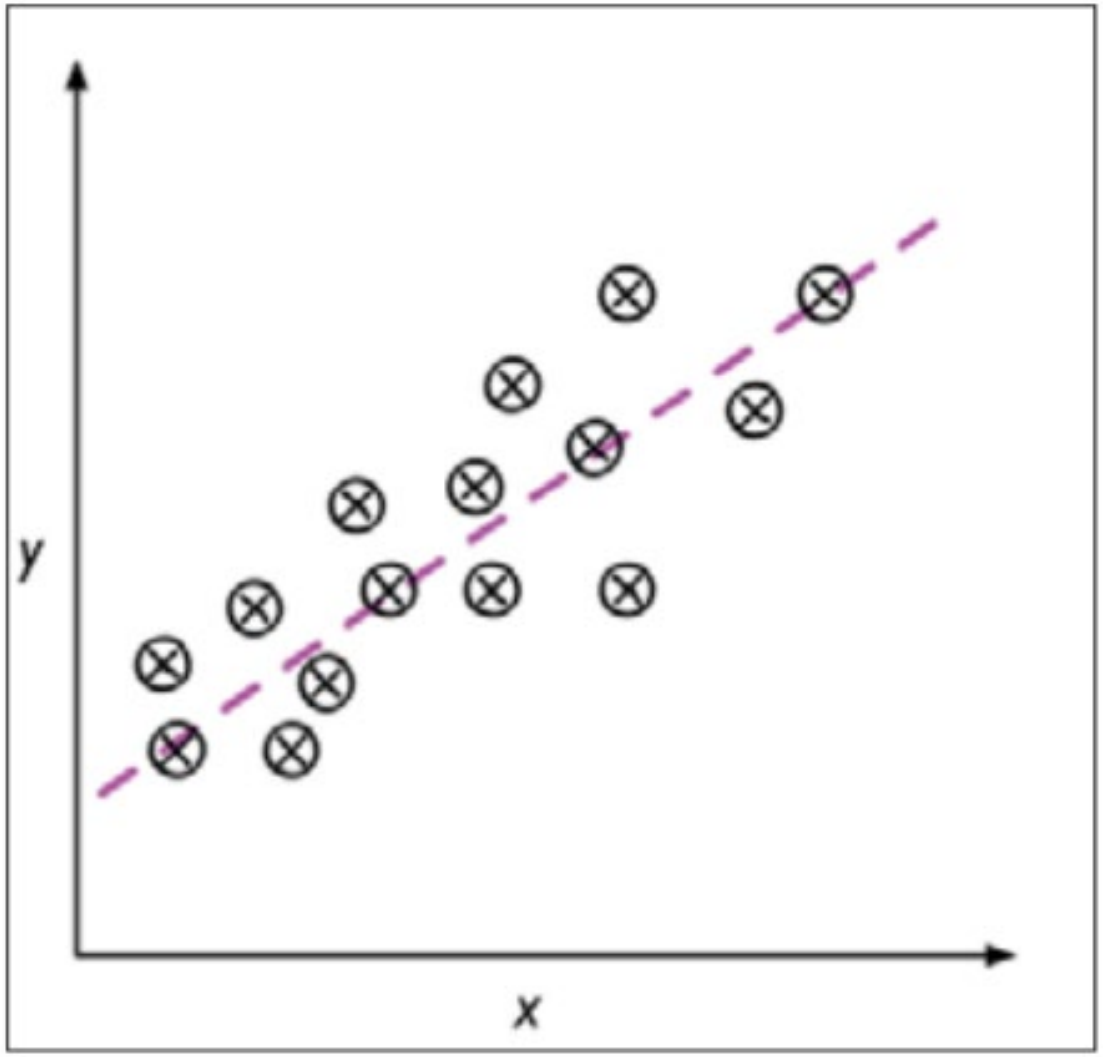
X Input	Y Output	Application
Email	Spam (0/1)	Spam Filtering
Audio	Text Transcripts	Speech Recognition
English	Hindi	Machine Traslation

Source: https://www.ejable.com/tech-corner/ai-machine-learning-and-deep-learning/types-of-machine-learning/

Classification for Predicting Class Labels

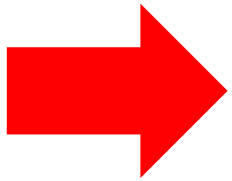


Regression for Predicting Continuous outcomes



Supervised Machine Learning Algorithms

Supervised Machine Learning Algorithms

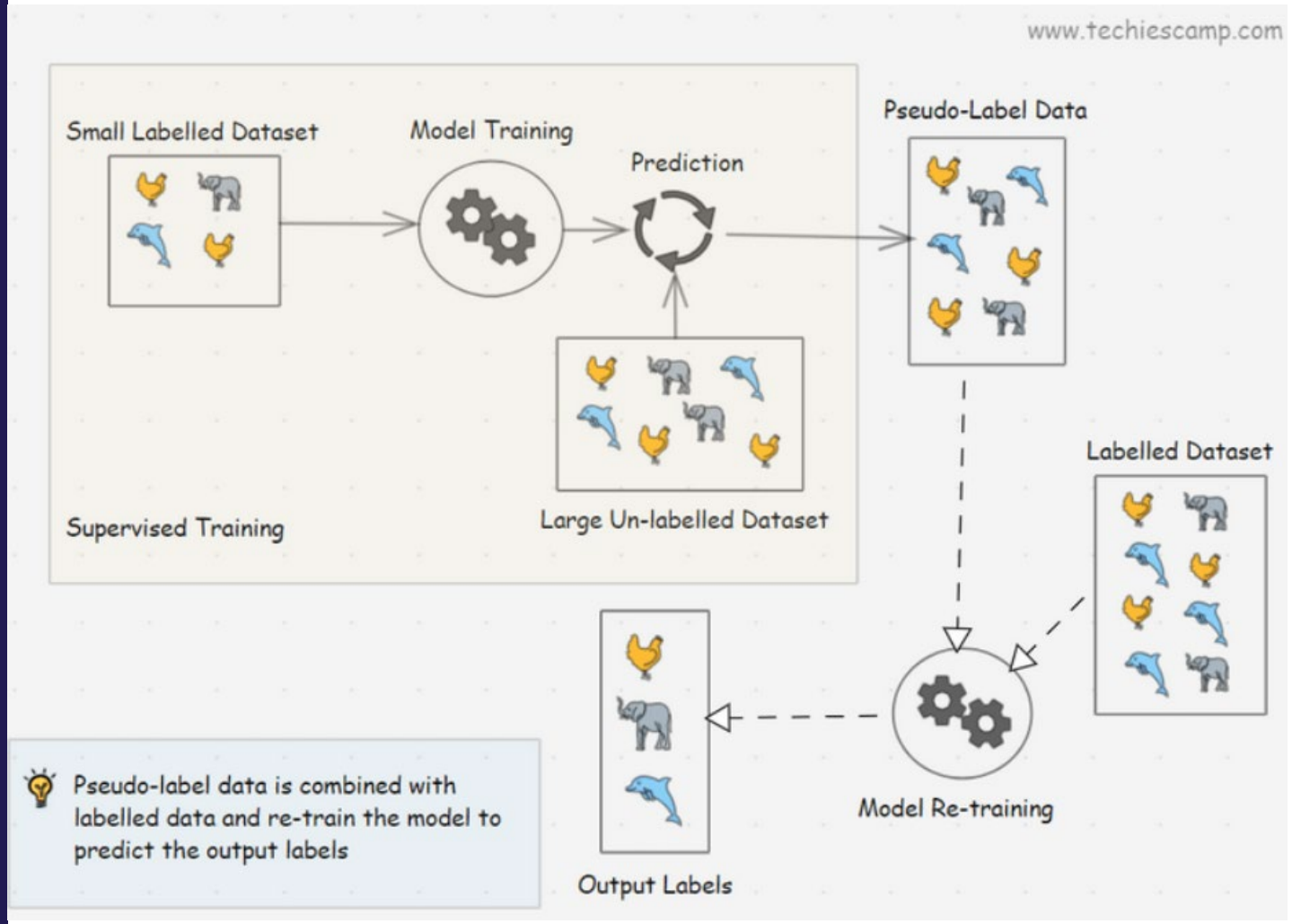


- Linear Regression
- Logistic Regression
- Decision Tree
- Random Forest
- (K-Nearest Neighbor) KNN
- Naïve Bayes
- Support vector machine

Semi-Supervised Learning

- **Semi-supervised learning** refers to a setting where only a small portion of the training samples are labeled, while a large amount of data remains unlabeled

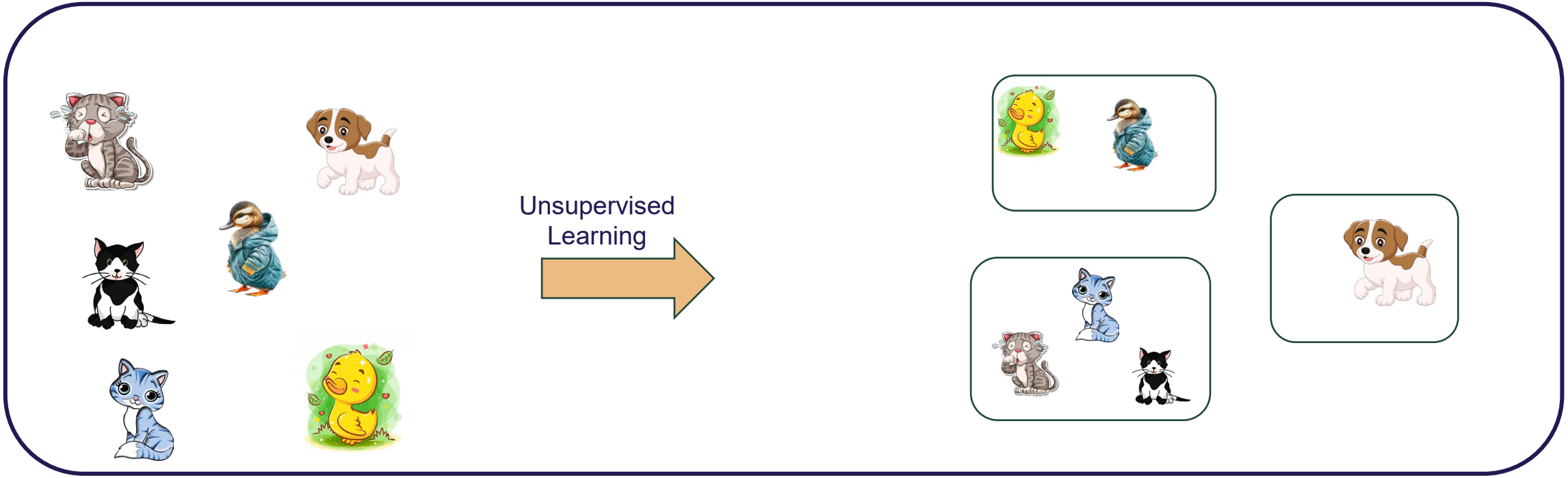
- **The main goal** is to leverage both labeled and unlabeled data to learn a model that makes better predictions on unseen or future data than using labeled data alone



Unsupervised Learning

- In unsupervised learning, we are dealing with unlabeled data or data of unknown structure

- Using unsupervised learning techniques, we are able to explore the structure of our data to extract meaningful information without the guidance of a known outcome variable



Clustering

Reinforcement Learning

- Another type of machine learning is reinforcement learning. In reinforcement learning, the goal is to develop a system (agent) that improves its performance based on interactions with the environment.

- Since the information about the current state of the environment typically also includes a so-called reward signal, we can think of reinforcement learning as a field related to the supervised learning

- However, in reinforcement learning this feedback is not the correct ground truth label or value, but a measure of how well the action was measured by a reward function.

- Through its interaction with the environment, an agent can then use reinforcement learning to learn a series of actions that maximizes this reward via an exploratory trial-and-error approach or deliberative planning

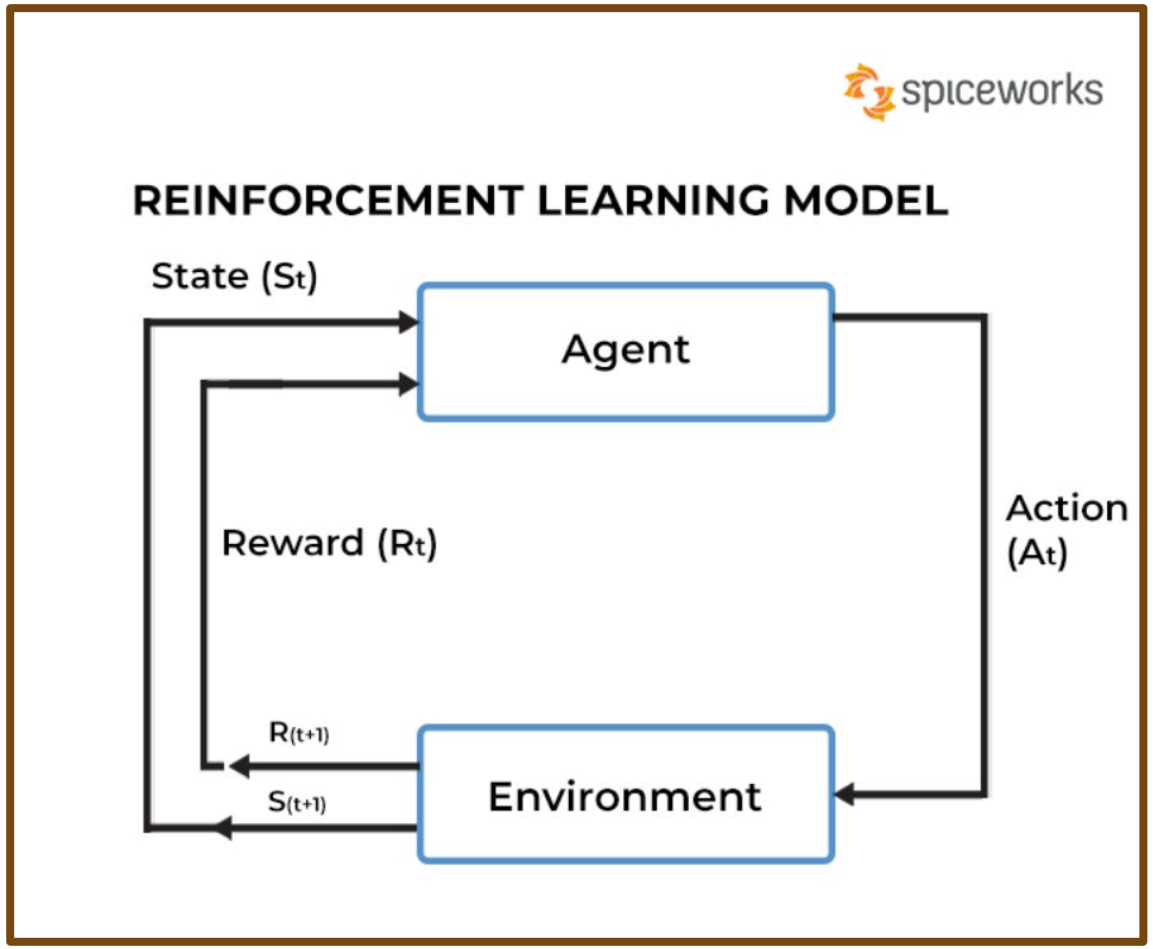


Image source: https://www.spiceworks.com/tech/artificial-intelligence/articles/what-is-reinforcement-learning/

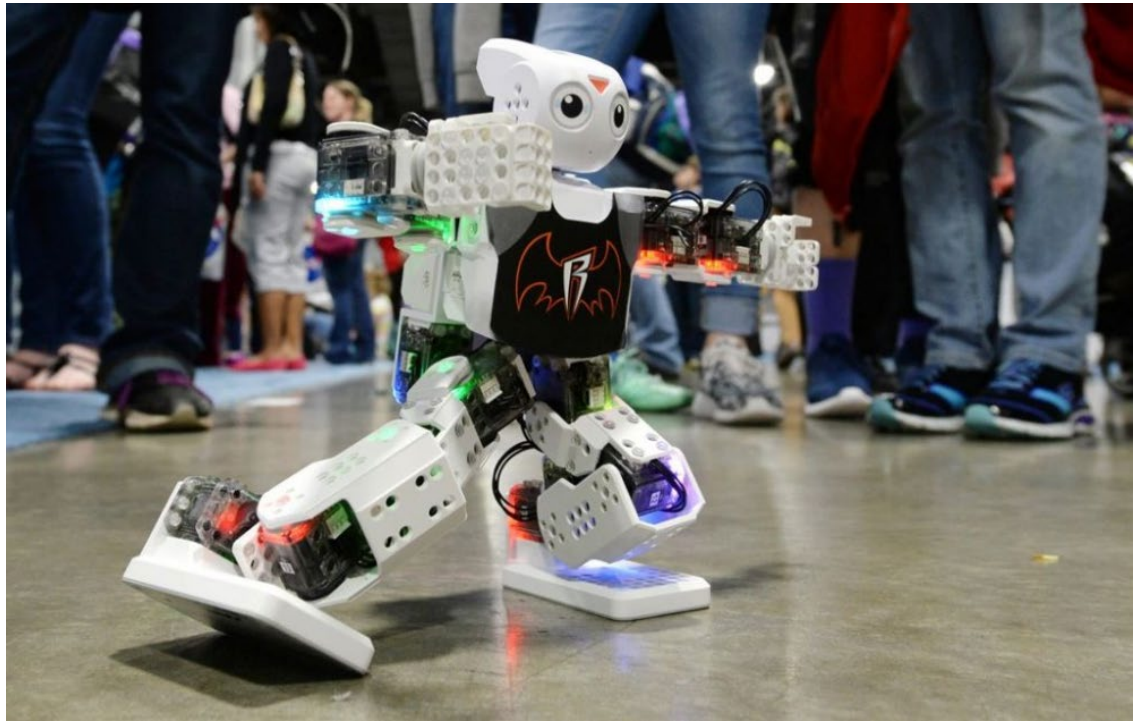
Chess Game (AlphaGo and AlphaZero)

- let's think of visiting certain locations on the chess board as being associated with a positive event—for instance, removing an opponent's chess piece from the board or threatening the queen

- Other positions, however, are associated with a negative event, such as losing a chess piece to the opponent in the following turn

Image source: https://www.freepik.com/premium-ai-image/exploring-battle-brains-humans-vs-robots-chess-ar-169_135996824.htm

RL in Robotics



Learning to Walk via Deep Reinforcement Learning:

https://www.youtube.com/watch?v=n2gE7n11h1Y

https://www.youtube.com/watch?v=m3wJGDiSxWY&t=4s

https://www.youtube.com/watch?v=LeeiN9smjjY

Image source: https://analyticsindiamag.com/ai-origins-evolution/reinforcement-learning-goes-beyond-gaming-robotics-will-become-a-game-changer-in-2019/

Tools used for Machine Learning

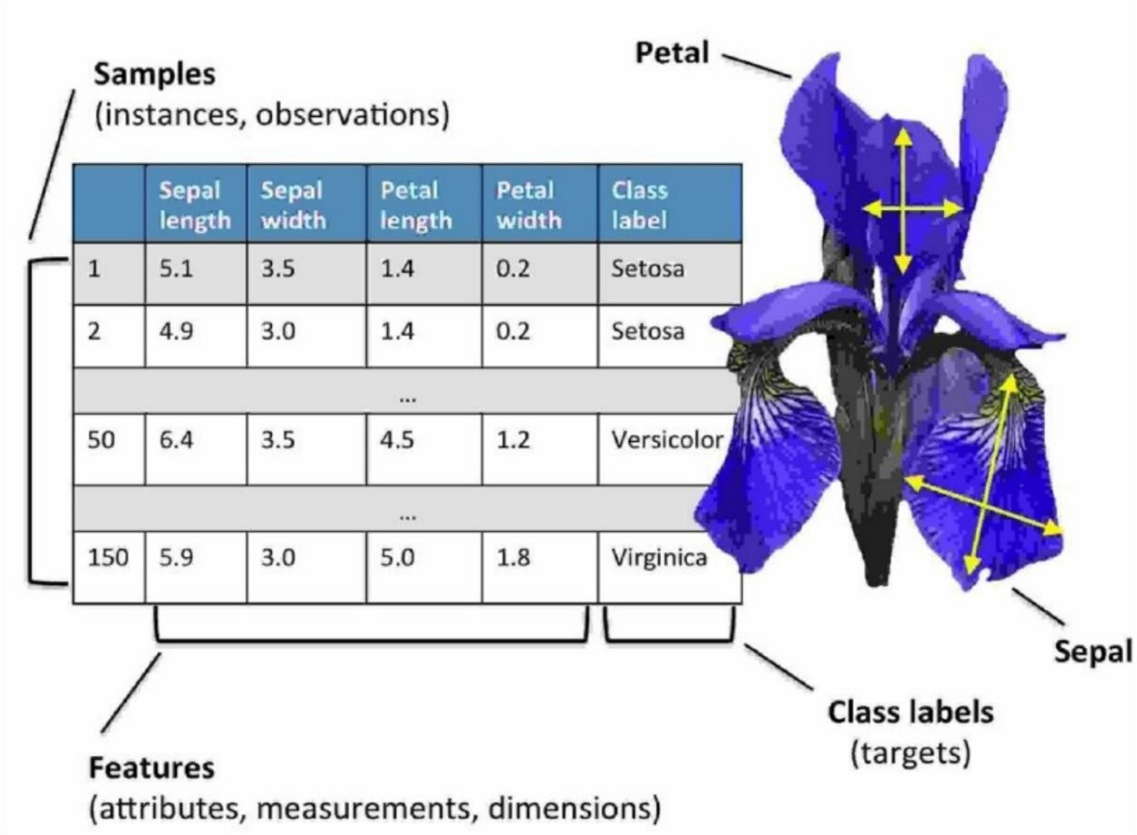
- WEKA
- Python
- Numpy
- Pandas
- Sklearn
- Tensorflow
- Hadoop
- Spark
- AWS
- GPUs, CPUs, TPUs, cognitive processors

Dataset Formats

.csv

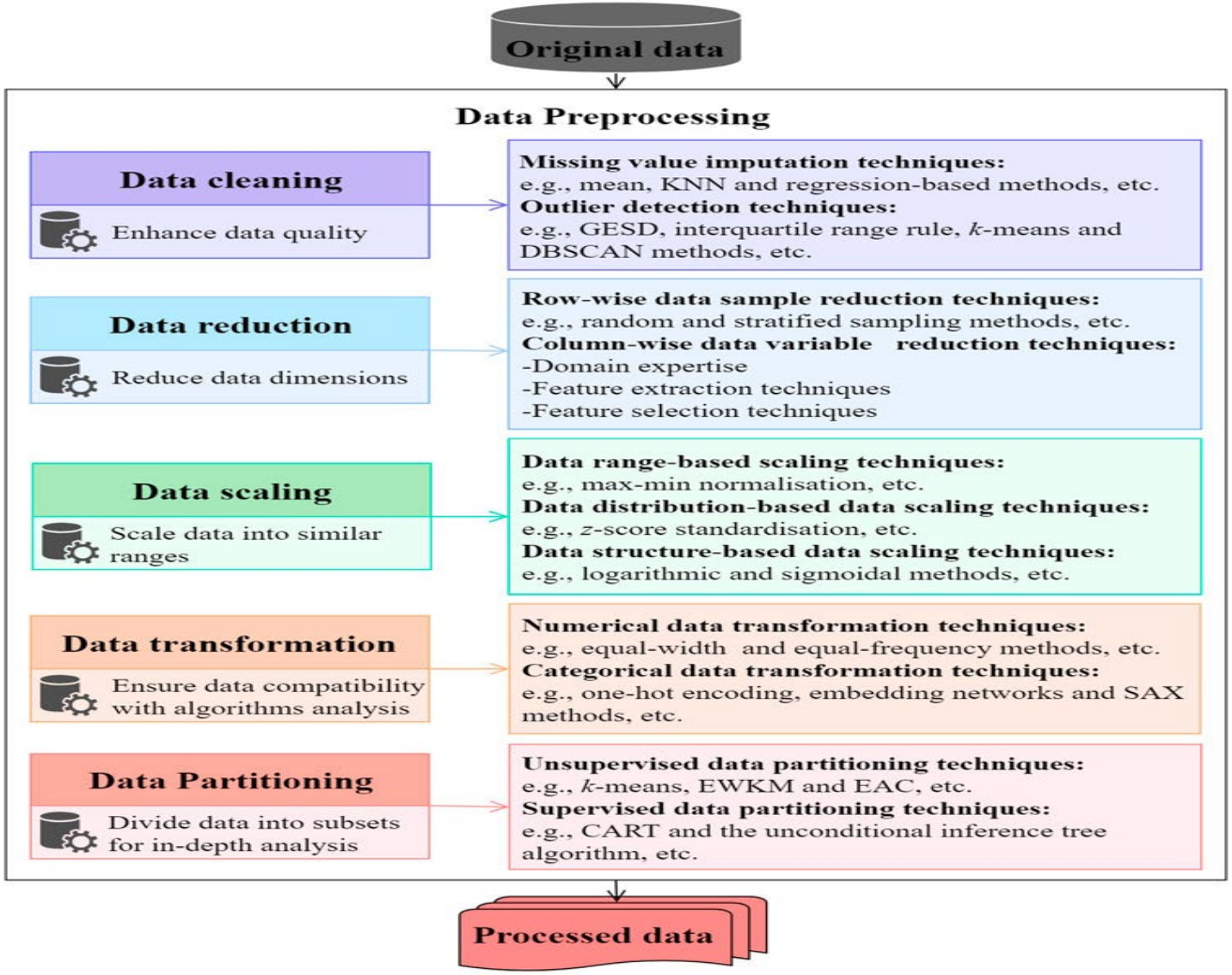
.libsvm

.arff



	Sepal length	Sepal width	Petal length	Petal width	Class label
1	5.1	3.5	1.4	0.2	Setosa
2	4.9	3.0	1.4	0.2	Setosa
...					
50	6.4	3.5	4.5	1.2	Versicolor
...					
150	5.9	3.0	5.0	1.8	Virginica

Data Pre-processing



Data Cleaning

- Samples to be missing one or more values for various reasons
- Most computational tools are unable to handle such missing values, or produce unpredictable result if we simply ignore them

```
>>> import pandas as pd
>>> from io import StringIO

>>> csv_data = \
... '''A,B,C,D
... 1.0,2.0,3.0,4.0
... 5.0,6.0,,8.0
... 10.0,11.0,12.0,'''
>>> # If you are using Python 2.7, you need
>>> # to convert the string to unicode:
>>> # csv_data = unicode(csv_data)
>>> df = pd.read_csv(StringIO(csv_data))
>>> df
     A    B    C    D
0 1.0  2.0  3.0  4.0
1 5.0  6.0  NaN  8.0
2 10.0 11.0 12.0 NaN
```


Image source: https://www.spiceworks.com/tech/artificial-intelligence/articles/what-is-reinforcement-learning/

Data Cleaning

- Eliminate the samples or features with missing values
- However, there is a problem when we eliminate rows or columns. What kind of problem? How to solve it?

```
>>> df.dropna(axis=0)
     A    B    C    D
0  1.0  2.0  3.0  4.0
```

```
>>> df.dropna(axis=1)
     A    B
0  1.0  2.0
1  5.0  6.0
2  10.0 11.0
```

Image source: https://www.spiceworks.com/tech/artificial-intelligence/articles/what-is-reinforcement-learning/

Imputing Missing Values

- the removal of samples or dropping of entire feature columns is simply not feasible, because we might lose too much valuable data
- In this case, we can use different interpolation techniques to estimate the missing values from the other training samples in our dataset
- One of the most common interpolation techniques is mean imputation, where we simply replace the missing value with the mean value of the entire feature column

```
>>> from sklearn.preprocessing import Imputer
>>> imr = Imputer(missing_values='NaN', strategy='mean', axis=0)
>>> imr = imr.fit(df.values)
>>> imputed_data = imr.transform(df.values)
>>> imputed_data
array([[  1.,   2.,   3.,   4.],
       [  5.,   6.,  7.5,   8.],
       [ 10.,  11.,  12.,   6.]])
```

- If we changed the axis=0 setting to axis=1, we'd calculate the row means
- Other options for the strategy parameter are median or most´frequent, where the latter replaces the missing values with the most frequent values.
- This is useful for imputing categorical feature values, for example, a feature column that stores an encoding of color names, such as red, green, and blue

Data Transformation

- For instance, **Categorial data**: **Nominal** and **Ordinal** features

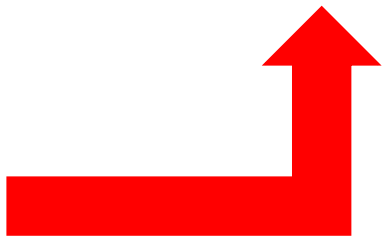
$$XL = L + 1 = M + 2$$

```
>>> import pandas as pd
>>> df = pd.DataFrame([
...             ['green', 'M', 10.1, 'class1'],
...             ['red', 'L', 13.5, 'class2'],
...             ['blue', 'XL', 15.3, 'class1']])
>>> df.columns = ['color', 'size', 'price', 'classlabel']
>>> df
   color size  price classlabel
0  green    M   10.1     class1
1    red    L   13.5     class2
2   blue   XL   15.3     class1
```

Nominal

Ordinal

```
>>> size_mapping = {
...                 'XL': 3,
...                 'L': 2,
...                 'M': 1}
>>> df['size'] = df['size'].map(size_mapping)
>>> df
   color  size  price classlabel
0  green     1   10.1     class1
1    red     2   13.5     class2
2   blue     3   15.3     class1
```



Encoding Class Labels

- Many machine learning libraries require that class labels are encoded as integer values

- Although most estimators for classification in scikit-learn convert class labels to integers internally, it is considered good practice to provide class labels as integer arrays to avoid technical glitches

```
>>> import numpy as np
>>> class_mapping = {label:idx for idx,label in
...                  enumerate(np.unique(df['classlabel']))}
>>> class_mapping
{'class1': 0, 'class2': 1}
```

```
>>> df['classlabel'] = df['classlabel'].map(class_mapping)
>>> df
   color  size  price  classlabel
0  green     1   10.1           0
1    red     2   13.5           1
2   blue     3   15.3           0
```

Performing one-hot encoding on nominal features

```
>>> X = df[['color', 'size', 'price']].values
>>> color_le = LabelEncoder()
>>> X[:, 0] = color_le.fit_transform(X[:, 0])
>>> X
array([[1, 1, 10.1],
       [2, 2, 13.5],
       [0, 3, 15.3]], dtype=object)
```

- blue = 0
- green = 1
- red = 2

```
>>> pd.get_dummies(df[['price', 'color', 'size']])
   price  size  color_blue  color_green  color_red
0   10.1     1           0            1          0
1   13.5     2           0            0          1
2   15.3     3           1            0          0
```

Performing one-hot encoding on nominal features

```
>>> X = df[['color', 'size', 'price']].values
>>> color_le = LabelEncoder()
>>> X[:, 0] = color_le.fit_transform(X[:, 0])
>>> X
array([[1, 1, 10.1],
       [2, 2, 13.5],
       [0, 3, 15.3]], dtype=object)
```

- blue = 0
- green = 1
- red = 2

```
>>> pd.get_dummies(df[['price', 'color', 'size']])
   price  size  color_blue  color_green  color_red
0   10.1     1           0            1          0
1   13.5     2           0            0          1
2   15.3     3           1            0          0
```

Feature/Data Scaling

- Normalization is also known as min-max normalization or min-max scaling
- Normalization re-scales values in the range of 0-1

$$x_{\text{norm}} = \frac{x - \min(x)}{\max(x) - \min(x)}$$

#	Emp	Age	Salary
1	Emp1	44	73000
2	Emp2	27	47000
3	Emp3	30	53000
4	Emp4	38	62000
5	Emp5	40	57000
6	Emp6	35	53000
7	Emp7	48	78000

Normalization

Age	Normalized Age	Salary	Normalized Salary
44	0.80952381	73000	0.838709677
27	0	47000	0
30	0.142857143	53000	0.193548387
38	0.523809524	62000	0.483870968
40	0.619047619	57000	0.322580645
35	0.380952381	53000	0.193548387
48	1	78000	1
	Range 0-1		**Range 0-1**

How to calculate Normalized value?
X = 35, min = 27, max = 48 for column Age.
Xnorm(for 35) = $\frac{35-27}{48-27} = 0.3809$

Feature/Data Scaling

- Standardization is also known as z-score Normalization
- In Standardization, features are scaled to have zero-mean and one-standard-deviation. It means after standardization features will have mean = 0 and standard deviation = 1

$$x_{\text{stand}} = \frac{x - \text{mean}(x)}{\text{standard deviation } (x)}$$

#	Emp	Age	Salary
1	Emp1	44	73000
2	Emp2	27	47000
3	Emp3	30	53000
4	Emp4	38	62000
5	Emp5	40	57000
6	Emp6	35	53000
7	Emp7	48	78000
		Mean = 37.42857 Std. Dev. = 6.883876	Mean = 60428.5714 Std. Dev = 10499.7570

Standardization

How to calculate Standardized value?
X = 35, mean = 37.42, Std. Dev. = 6.88
for column Age.
Xstd(for 35) = $\frac{35-37.42}{6.88}$ = -0.3527

Age	Standardized Age	Salary	Standardized Salary
44	0.954611636	73000	1.197306616
27	-1.514927162	47000	-1.278941158
30	-1.079126198	53000	-0.707499364
38	0.083009708	62000	0.149663327
40	0.373543684	57000	-0.326538168
35	-0.352791257	53000	-0.707499364
48	1.535679589	78000	1.673508111
	Mean = 0 Std. dev. = 1		**Mean = 0 Std. dev. = 1**

Feature Selection (Data Reduction)

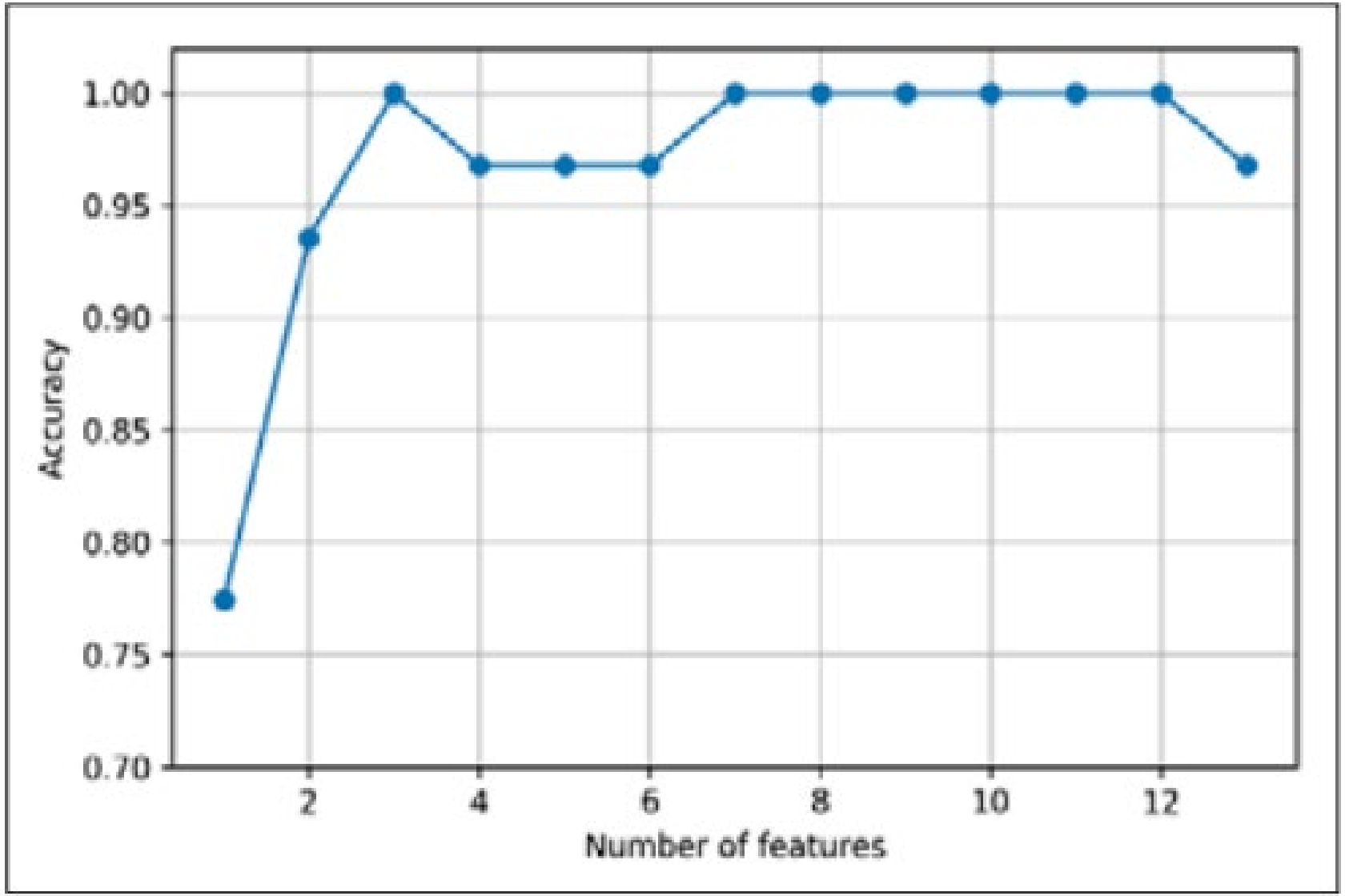
- An alternative way to reduce the complexity of the model and avoid overfitting is dimensionality reduction via feature selection, which is especially useful for unregularized models

- There are two main categories of dimensionality reduction techniques: feature selection and feature extraction

Sequential Backward Selection (SBS)

1. Initialize the algorithm with $k=d$, where d is the dimensionality of the full feature space $\mathbf{X}_d$.
2. Determine the feature x^- that maximizes the criterion: $\boldsymbol{x}^- = \text{argmax}\, J(\boldsymbol{X}_k - \boldsymbol{x}))$ where $\boldsymbol{x} \in \boldsymbol{X}_k$.
3. Remove the feature x^- from the feature set: $\boldsymbol{X}_{k-1} = \boldsymbol{X}_k - \boldsymbol{x}^-; k = k-1$.
4. Terminate if k equals the number of desired features; otherwise, go to step 2.

Feature Selection



Feature Extraction

- Principal Component Analysis (PCA) for unsupervised data compression
- Linear Discriminant Analysis (LDA) as a supervised dimensionality reduction technique for maximizing class reparability

Principal Component Analysis (PCA)

		1	2	3	4	5	6	7		200
	Lifespan	Height	Weight	Average blood pressure	Average heart rate	BMI	Cholesterol levels	Average cigarettes/day	...	Sugar levels
Person 1	82	150	80	140/90	63	36	5.0	0		99
Person 2	73	174	90	90/60	100	32	4.1	0		95
Person 3	95	183	109	120/80	95	29	3.6	1		92
Person 4	92	186	95	123/75	84	28	4.8	5		89
Person 5	87	170	67	95/60	76	23	2.7	10		100
Person 6	65	180	82	92/60	78	25	3.7	10		112
Person 7	93	165	71	124/80	81	26	3.8	0		113
Person 8	80	172	70	97/70	90	24	3.4	0		100
...										
Person 20	72	190	75	90/60	78	21	4.2	0		82

Source: Principal Component Analysis (PCA) - easy and practical explanation - YouTube

Feature Extraction

1	2	3	4	5	6	7	...	200
Height	Weight	Average blood pressure	Average heart rate	BMI	Cholesterol levels	Average cigarettes/day	...	Sugar levels
150	80	140/90	63	36	5.0	0		99
174	90	90/60	100	32	4.1	0		95
183	109	120/80	95	29	3.6	1		92
186	95	123/75	84	28	4.8	5		89
170	67	95/60	76	23	2.7	10		100
180	82	92/60	78	25	3.7	10		112
165	71	124/80	81	26	3.8	0		113
172	70	97/70	90	24	3.4	0		100
190	75	90/60	78	21	4.2	0		82

Principal Component Analysis

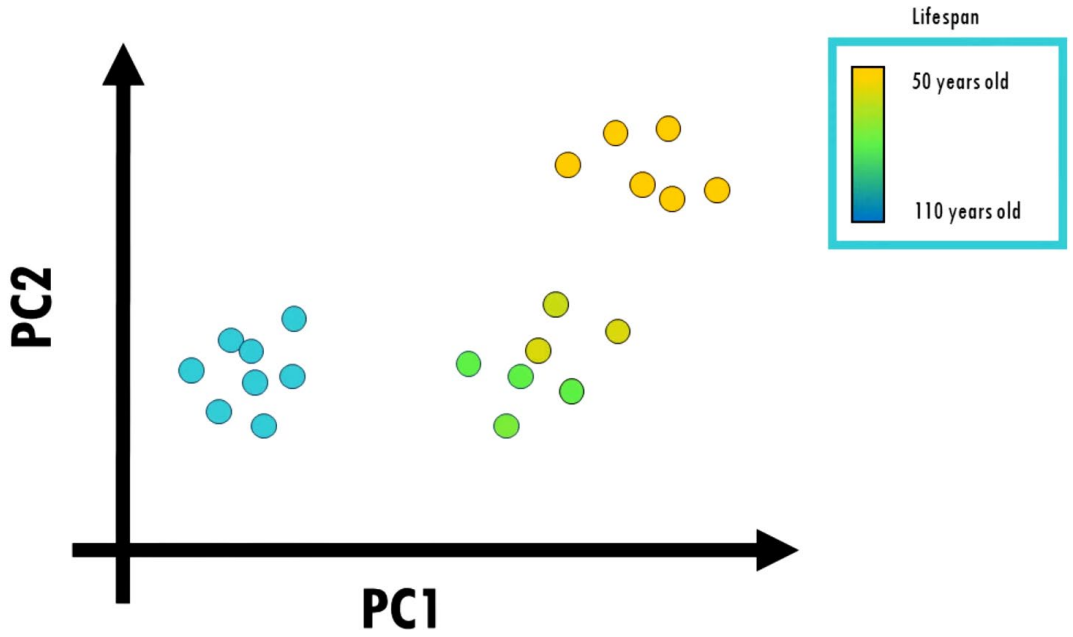
PC1	PC2	PC3	PC4	PC5
-1	3	-1	4	4
2	4	2	5	5
3	2	4	2	2
4	4	5	-4	-4
5	5	2	2	5
2	5	-4	3	2
-4	-6	5	5	-4
-3	-6	-6	2	5
8	-3	-6	-3	-6

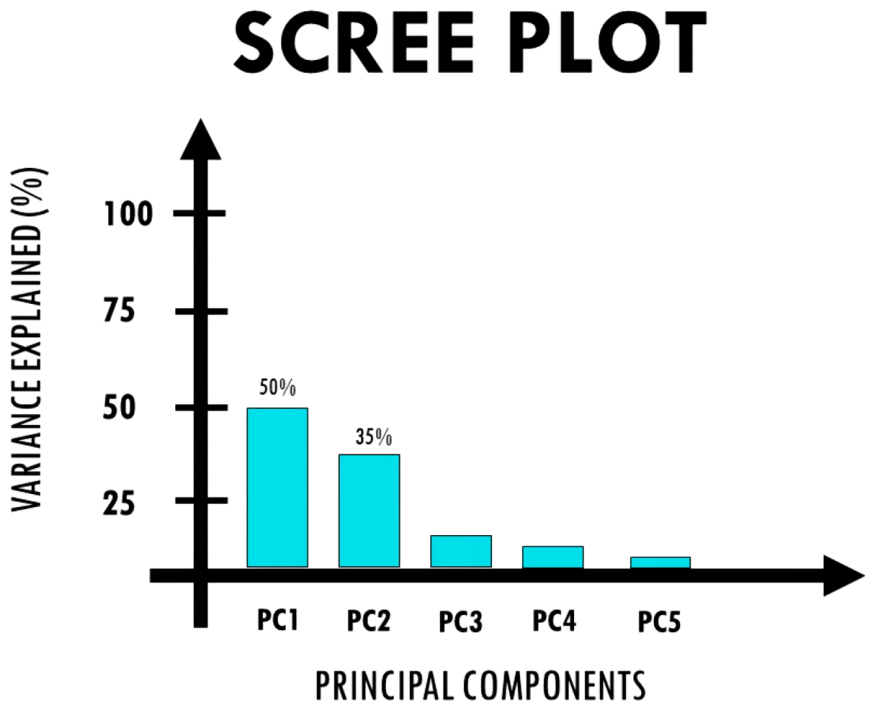
- It is not possible to express all PCs in 2D plot
- However, principle components are ranked:

PC1 > PC2 > PC3 > PC4 > PC5

Source: Principal Component Analysis (PCA) - easy and practical explanation - YouTube

Feature Extraction





- Ideally, we want to get around 90% variance with just 2 to 3 PCs so that enough information is retained while we can still visualize our data on a plot

Source: Principal Component Analysis (PCA) - easy and practical explanation - YouTube

Feature Extraction

1. Standardize the d-dimensional dataset

```
>>> # standardize the features
>>> from sklearn.preprocessing import StandardScaler
>>> sc = StandardScaler()
>>> X_train_std = sc.fit_transform(X_train)
>>> X_test_std = sc.transform(X_test)
```

2. Construct the covariance matrix

```
>>> import numpy as np
>>> cov_mat = np.cov(X_train_std.T)
>>> eigen_vals, eigen_vecs = np.linalg.eig(cov_mat)
>>> print('\nEigenvalues \n%s' % eigen_vals)
Eigenvalues
[ 4.84274532  2.41602459  1.54845825  0.96120438  0.84166161
0.6620634   0.51828472  0.34650377  0.3131368   0.10754642
0.21357215  0.15362835  0.1808613 ]
```

3. Decompose the covariance matrix into its eigenvectors and eigenvalues

4. Sort the eigenvalues by decreasing order to rank the corresponding eigenvectors

```
>>> # Make a list of (eigenvalue, eigenvector) tuples
>>> eigen_pairs = [(np.abs(eigen_vals[i]), eigen_vecs[:, i])
...                for i in range(len(eigen_vals))]
>>> # Sort the (eigenvalue, eigenvector) tuples from high to low
>>> eigen_pairs.sort(key=lambda k: k[0], reverse=True)
```

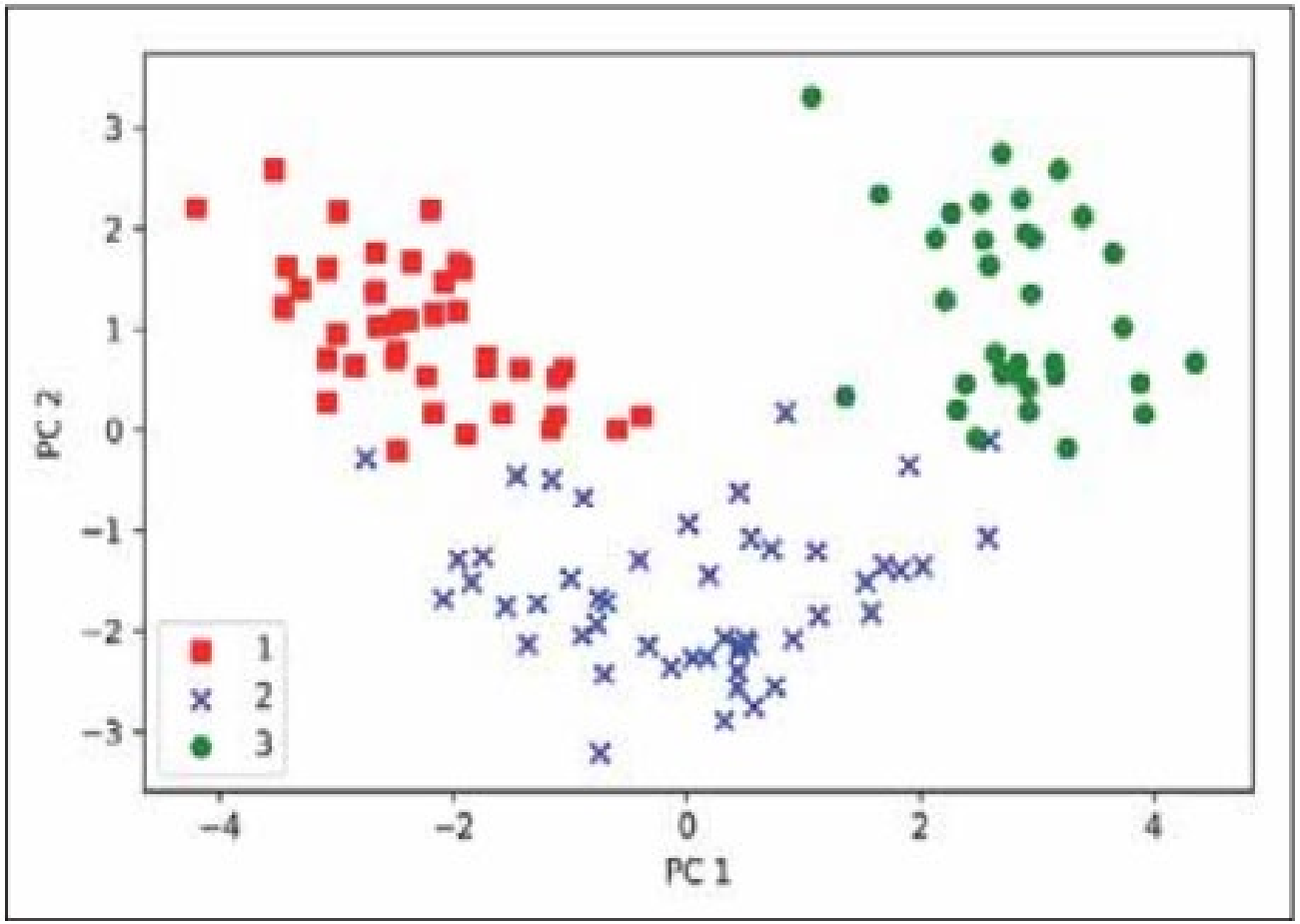
5. Select N eigenvectors which correspond to the N largest eigenvalues, where N is the dimensionality of the new feature subspace ($k < d$)

Feature Extraction

6. Construct a projection matrix W from the "top" N eigenvectors

```
>>> # standardize the features
>>> from sklearn.preprocessing import StandardScaler
>>> sc = StandardScaler()
>>> X_train_std = sc.fit_transform(X_train)
>>> X_test_std = sc.transform(X_test)
```

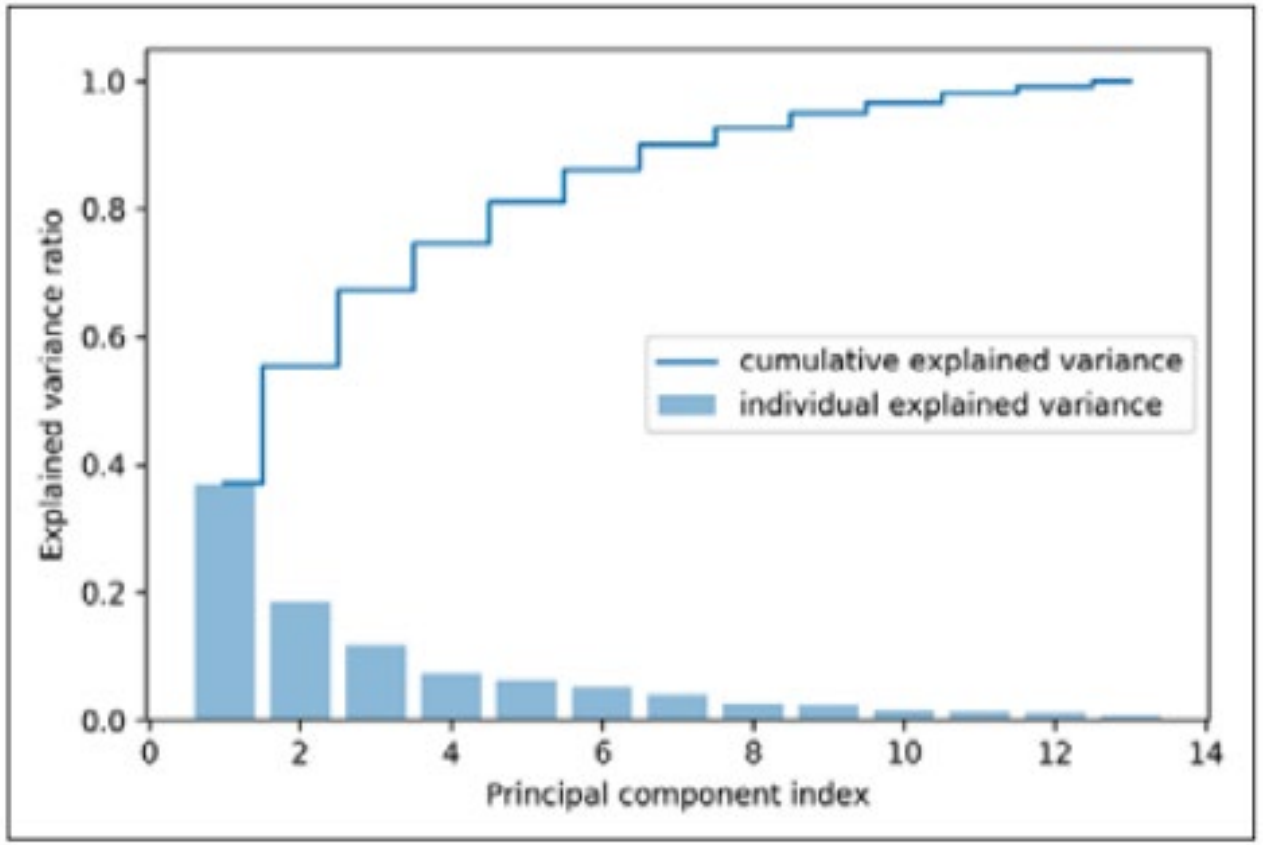
```
>>> w = np.hstack((eigen_pairs[0][1][:, np.newaxis],
...                eigen_pairs[1][1][:, np.newaxis]))
>>> print('Matrix W:\n', w)
Matrix W:
 [[-0.13724218  0.50303478]
 [ 0.24724326  0.16487119]
 [-0.02545159  0.24456476]
 [ 0.20694508 -0.11352904]
 [-0.15436582  0.28974518]
 [-0.39376952  0.05080104]
 [-0.41735106 -0.02287338]
 [ 0.30572896  0.09048885]
 [-0.30668347  0.00835233]
 [ 0.07554066  0.54977581]
 [-0.32613263 -0.20716433]
 [-0.36861022 -0.24902536]
 [-0.29669651  0.38022942]]
```



Total and explained variance

$$\frac{\lambda_j}{\sum_{j=1}^{d} \lambda_j}$$

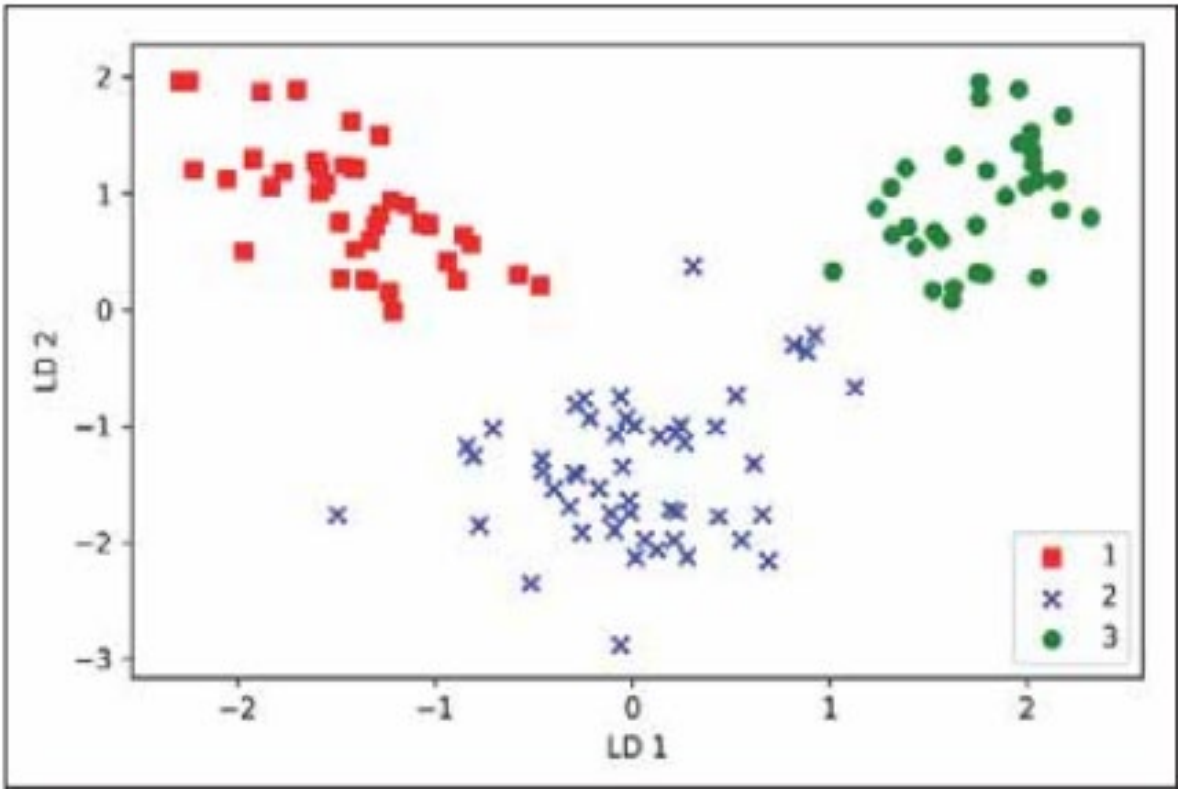
λ j – eigenvalue

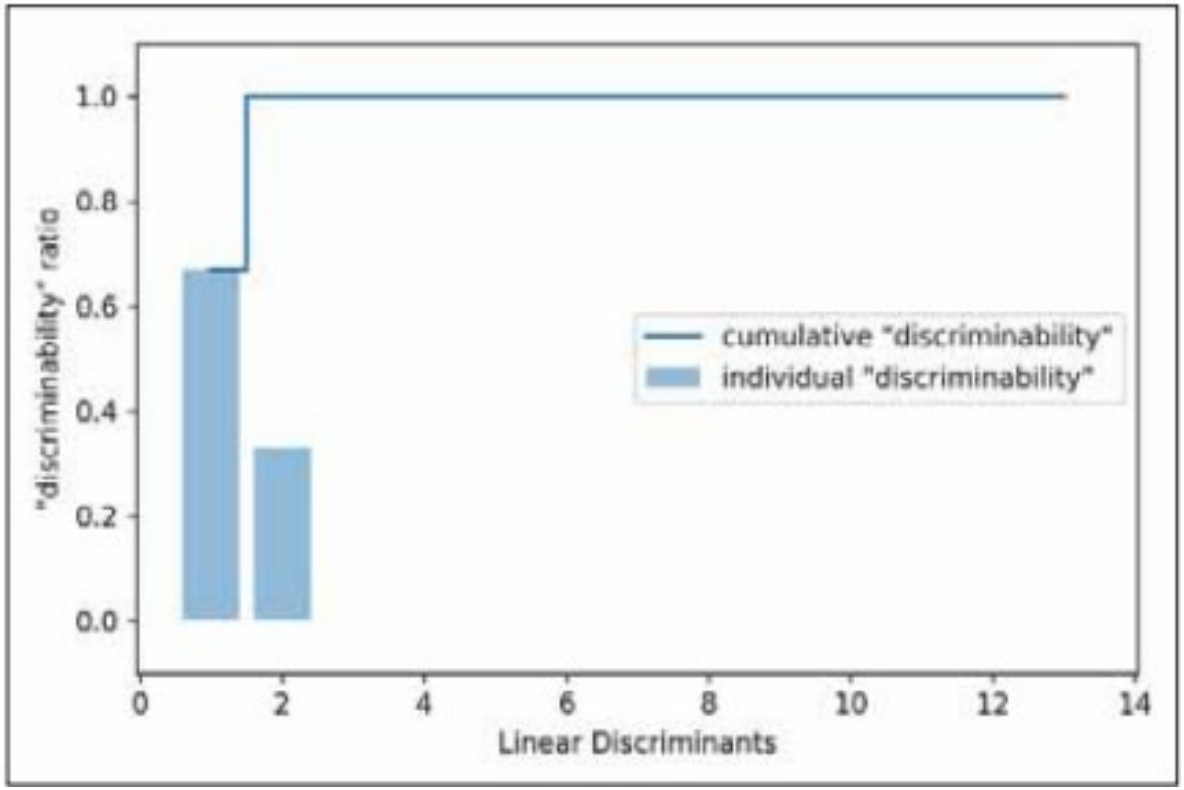


Linear Discriminant Analysis

1. Standardize the d-dimensional dataset (d is the number of features)
2. For each class, compute the d-dimensional mean vector
3. Construct the between-class scatter matrix S_B and the within-class scatter matrix S_w
4. Compute the eigenvectors and corresponding eigenvalues of the matrix $S_w^{-1}S_B$
5. Sort the eigenvalues by decreasing order to rank the corresponding eigenvectors
6. Choose the N eigenvectors that correspond to the N largest eigenvalues to construct a d k × -dimensional transformation matrix W; the eigenvectors are the columns of this matrix

Linear Discriminant Analysis





Data Partitioning

Dividing data into training and test data

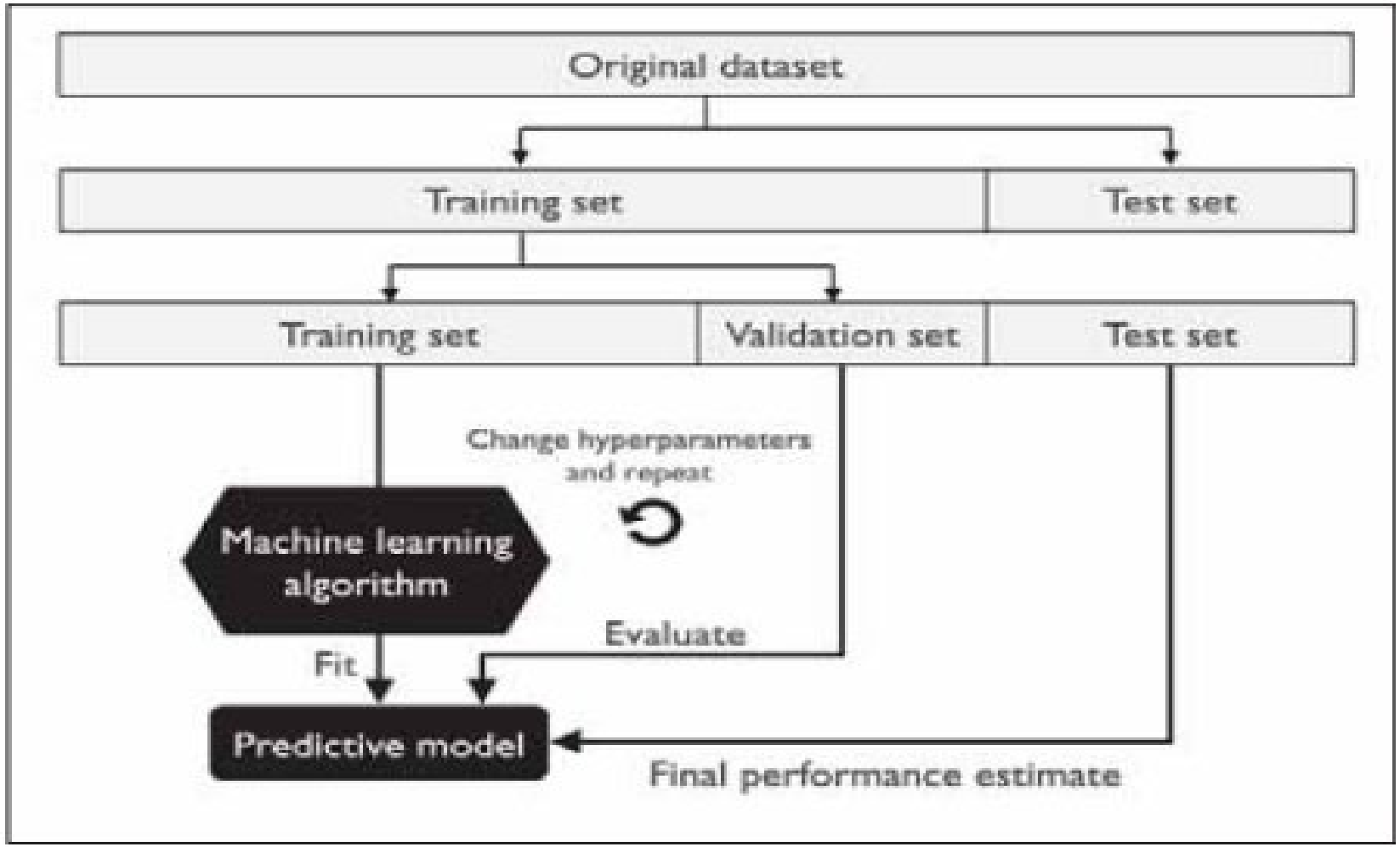
```
>>> from sklearn import datasets
>>> import numpy as np

>>> iris = datasets.load_iris()
>>> X = iris.data[:, [2, 3]]
>>> y = iris.target
>>> print('Class labels:', np.unique(y))
Class labels: [0 1 2]
```

Import Iris dataset

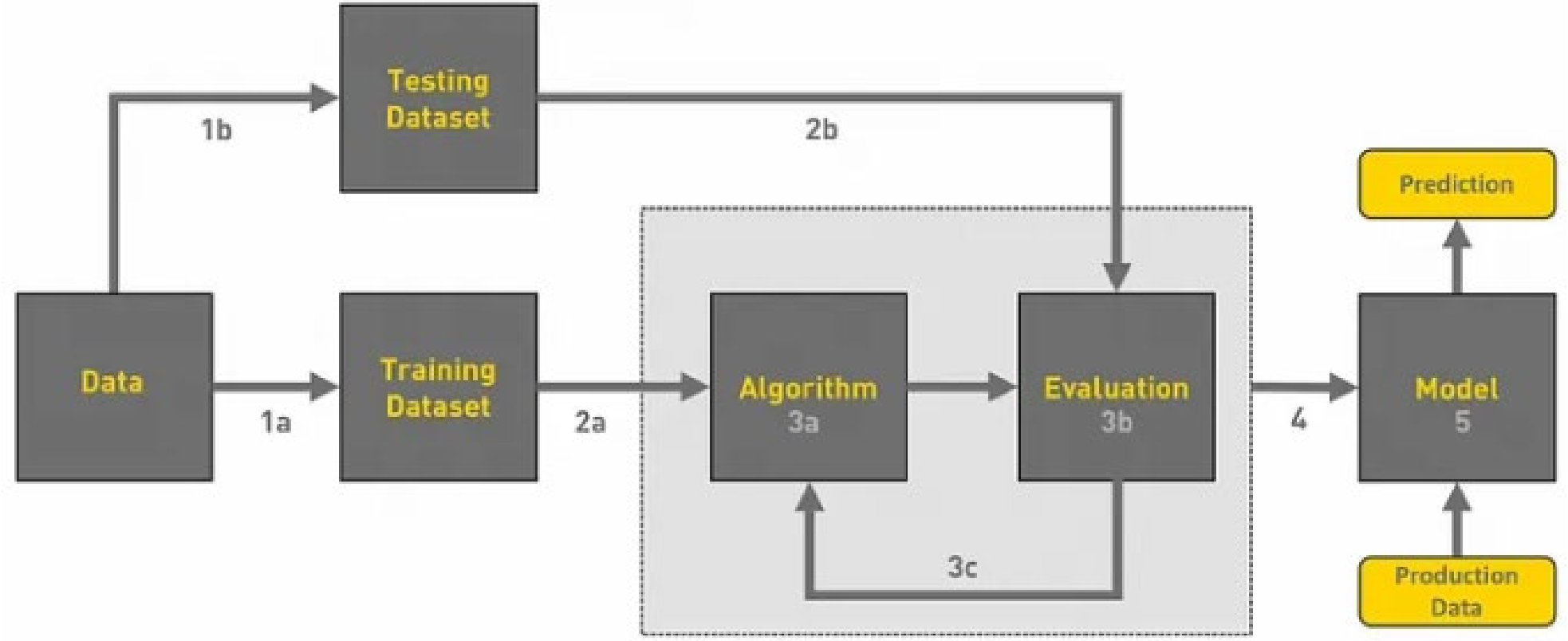
```
>>> from sklearn.model_selection import train_test_split
>>> X_train, X_test, y_train, y_test = train_test_split(
...     X, y, test_size=0.3, random_state=1, stratify=y)
```

```
>>> print('Labels counts in y:', np.bincount(y))
Labels counts in y: [50 50 50]
>>> print('Labels counts in y_train:', np.bincount(y_train))
Labels counts in y_train: [35 35 35]
>>> print('Labels counts in y_test:', np.bincount(y_test))
Labels counts in y_test: [15 15 15]
```

Data Partitioning



Work Flow of Machine Learning



Overview of the Workflow of ML

Source: https://towardsdatascience.com/workflow-of-a-machine-learning-project-ec1dba419b94


Q&A